

Representations of an Operating System Process

Group Members

August 31, 2026

Abstract

This document outlines the core processes of an operating system (OS), detailing the boot sequence, hardware-software interfacing, and kernel resource management. By employing diverse representational formats—including mathematical modeling, algorithmic pseudo-code, and structural diagrams—it provides a comprehensive breakdown of how an OS initializes and maintains a stable, secure environment for user applications.

1 Introduction

An operating system (OS) acts as the foundational software layer that manages and controls all hardware and software components within a computer [1]. Its startup sequence begins with a small program called a bootloader, which initializes the hardware and loads the core of the OS—the kernel—into main memory.

Once activated, the kernel assumes full control of the system, efficiently allocating CPU time, managing memory processes, coordinating input/output devices, and providing a stable, secure environment where user applications can seamlessly execute.

2 Technical Background

To understand how an operating system functions, it is essential to define its core hardware and software components, as well as the mathematical state-transition models that dictate its continuous operation and efficiency [2].

The comparison presented in Table 1 indicates the specific execution phases and primary functions of each core system component.

Table 1: System Component and Execution Phase

System Component	Primary Function	Execution Phase
Firmware (BIOS/UEFI)	Performs hardware checks (POST) and finds the boot device	Immediate Power-On
Bootloader	Loads the kernel into main memory	Early Boot Sequence
Kernel	Manages CPU, RAM, and hardware resource allocation	Continuous Operation
Shell / User Space	Provides the interface for user applications to run	Post-Boot / Active

2.1 Mathematical Models of the OS

The overall operation of the kernel can be abstracted as an event-driven state machine, mathematically defined in Equation 1.

$$S_{t+1} = k \left(S_t, \sum_{i=1}^h I_i(t) + \sum_{j=1}^s C_j(t) \right) - \Delta_{overhead} \quad (1)$$

Interpretation of Equation 1: This equation demonstrates that the OS continuously evaluates the current state of the machine (S_t) alongside a stream of hardware interrupts (ΣI) and software system calls (ΣC). It calculates the next system state (S_{t+1}) while deducting the necessary processing cost ($\Delta_{overhead}$).

- S_{t+1} (**Next System State**): The resulting state of the computer (e.g., updated memory allocation, next thread in the CPU).
- k (**The Kernel Function**): The core operating system logic.
- S_t (**Current System State**): The exact configuration of the CPU registers, RAM, and I/O devices at the current moment in time.
- $\Sigma I_{i(t)}$ (**Hardware Interrupts**): The sum of all hardware signals happening at time t .
- $\Sigma C_{j(t)}$ (**System Calls**): The sum of all software requests from user applications at time t .
- $\Delta_{\{overhead\}}$ (**Context Switching Overhead**): The inevitable computational "tax" the OS consumes to manage these transitions.

Furthermore, the efficiency of the operating system's process management can be analyzed using CPU scheduling metrics, such as process turnaround time, as defined in Equation 2.

$$T_{turnaround} = T_{completion} - T_{arrival} \quad (2)$$

Interpretation of Equation 2: Equation 2 measures the total time taken to execute a process from its initial submission to its final completion. By minimizing $T_{turnaround}$ for concurrent applications, the OS kernel ensures responsive memory management and optimal CPU allocation, preventing system bottlenecks.

3 Analysis/Working

The practical execution of an OS begins with a strict chronological boot sequence and relies on a layered architectural structure to separate user tasks from critical hardware functions [3].

Chronological Boot Sequence

1. The physical machine is powered on, sending a wake signal to the CPU.
2. The motherboard firmware (BIOS/UEFI) executes a hardware diagnostic check.
3. The firmware locates the bootable storage drive.
4. The system executes the bootloader (such as GRUB or Windows Boot Manager).
5. The bootloader transfers the core OS kernel into the system's memory.

6. Kernel takes over hardware management, starts services, and presents the login screen to user.

Algorithm 1 OS Boot Sequence

```

1: START OS_Boot_Sequence
2: Initialize CPU and apply power
3: Run Power-On Self Test (POST)
4: if hardware fails POST then
5:   HALT and display error code
6: else
7:   Identify primary boot device (SSD/HDD/Network)
8:   Load Bootloader from EFI System Partition / MBR
9:   Execute Bootloader
10:  Bootloader decompresses and loads Kernel into RAM
11:  Pass system control to Kernel
12:  Kernel initializes device drivers and memory structures
13:  Kernel mounts the root file system
14:  Kernel starts background system services (daemons)
15:  Launch Graphical User Interface (GUI) or Command Line Interface (CLI)
16: end if
17: END OS_Boot_Sequence

```

The computational steps are illustrated in Figure 1.

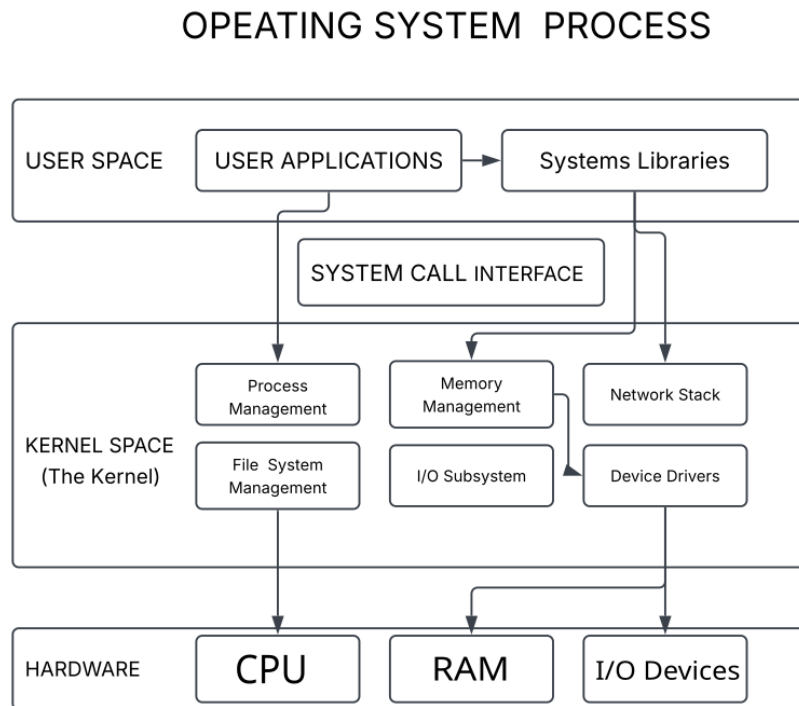


Figure 1: Operating System Architecture Layers

4 Discussion

The representations above highlight the immense complexity hidden beneath a standard computing interface. To demystify the algorithmic logic presented in the working section, the OS boot sequence can be understood through this simple explanation:

When we switch on the computer, the CPU starts, and the system first performs POST, which checks whether the basic hardware is working properly.

- **If hardware fails** → the system stops and shows an error.
- **If everything is fine** → the system finds the boot device, like an SSD/HDD.

It then loads and runs the bootloader. The bootloader loads the OS kernel into RAM and gives control to it. The kernel initializes drivers, memory, and the root filesystem. Then it starts necessary background services (daemons). Finally, the system starts the GUI or CLI, allowing the user to interact with the computer.

Beyond the boot sequence, the core responsibilities derived from the OS architecture include:

- **Process Management:** Creating, suspending, scheduling, and terminating software processes.
- **Memory Management:** Tracking available RAM and securely allocating it to active applications.
- **File System Management:** Organizing directories, writing data to storage, and enforcing file permissions.
- **Device Management:** Using drivers to facilitate seamless communication between the software and physical peripherals.

5 Conclusion

The operating system is a highly orchestrated, event-driven resource manager that bridges the gap between physical hardware limitations and user application demands. Through a highly structured boot sequence and strict architectural layering, the OS successfully provides a continuous, stable, and secure computing environment.

References

- [1] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.
- [2] Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, 4th ed. Pearson, 2014.
- [3] Robert Love, *Linux Kernel Development*, 3rd ed. Addison-Wesley Professional, 2010.